SQUARE MATRIX MULTIPLICATION

Sofiia Chorna
ENSTA Paris
{sofiia-chorna@ensta-paris.fr}

ABSTRACT

As part of the practical task, we implemented matrix multiplication in three versions. Firstly, the basic GPU version was created, where multiplication is done using rows and columns in threads using the global memory of the GPU. Secondly, a tiled multiplication was added where the thread uses a shared memory to calculate the values in the tile. Finally, we used *cuBLAS* library to process matrix multiplication. The experiments were conducted with a range of tile dimensions to compare the execution time for the tiled version. We could conclude that the *cuBLAS* implementation provides the most optimal execution time. The code is available at this link.

1 Problem definition

Matrix multiplication can be expressed as follows:

$$C = \alpha \cdot A \cdot B + \beta \cdot C$$

where A is an $m \times k$ matrix, B is an $k \times n$ matrix, C is an $m \times n$ matrix, and α and β are scalar coefficients. The elements of these matrices are defined as:

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1k} \\ a_{21} & a_{22} & \cdots & a_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mk} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{k1} & b_{k2} & \cdots & b_{kn} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{m1} & c_{m2} & \cdots & c_{mn} \end{bmatrix}$$

Each element c_{ij} in the resulting matrix is computed as:

$$c_{ij} = \alpha \sum_{k=1}^{n} a_{ik} b_{kj} + \beta c_{ij}$$

2 Implementations

We implemented matrix multiplication using three approaches: basic CUDA-based version, tiled shared memory version and cuBLAS-based version. To evaluate performance, we compared the execution times against a CPU-based version, which serves as a reference for speedup calculations. On average, CPU implementation takes 16541.49 milliseconds to compute multiple matrices of size 32×40.

2.1 Basic version

Firslty, we created a basic GPU implementation using global memory. Each block is limited to 1024 threads, and the matrix tiles are of size 32×32. The total matrix dimensions are set by multiplying the tile width by a size of 40, resulting in square matrices of size 1280×1280.

In this approach, each thread computes a single element of matrix C by accessing a row of matrix A and a column of matrix B [1]. We have executed this implementation five times and took the average execution time. The results are presented in Table 1.

Run 1	Run 2	Run 3	Run 4	Run 5	**Avg**
171.97	182.78	185.85	166.35	171.63	**175.71**

Table 1: Execution times for basic matrix multiplication using CUDA over multiple runs

So, compared to the CPU version, basic GPU matrix multiplication has a speedup of around 94x. We could explain this acceleration due to the ability of the GPU to handle thousands of threads concurrently.

2.2 Tiled version

The tiled algorithm performs matrix multiplication using sub-matrix blocks that fit into shared memory [1]. In contrast to the previous version, where each row and column vector generates a single output value, the tiled matrix multiplication approach generates multiple output values for each row and column vector. This reduces memory latency thanks to the GPU's shared memory.

Table 2 presents the execution results for the tiled GPU-based version, using a square tile size of 32×32, with the total matrix dimensions resulting in square matrices of size 1280×1280.

Tile size	Run 1	Run 2	Run 3	Run 4	Run 5	Avg
8	15.27	17.29	23.45	23.44	21.80	**20.25**
16	18.96	20.31	12.28	21.29	13.85	**17.33**
32	11.62	9.81	9.64	11.98	13.07	**11.22**

Table 2: Execution times for tiled matrix multiplication using CUDA over multiple runs and varying the size of the tile

We observe an interesting pattern: increasing the tile size improves performance. Let's see precisely why.

Given matrices $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$, each $b \times b$ block needs to load $2b^2$ values (from A and B), and there are $\frac{m}{b} \times \frac{n}{b}$ blocks in total. This results in:

$$\text{Total memory fetches} = \left(\frac{m}{b} \times \frac{n}{b} \times k\right) \times 2b^2 = 2 \cdot \frac{m \cdot n \cdot k}{b}$$

Thus, using a tile of size $b \times b$ reduces the total number of memory accesses by a factor of b. However, there should be a limit of this pattern with the too big values of tile width.

So, the tiled version of size 32 has a speedup 11.61 of comparing to the basic GPU matrix multiplication, and 1099 for the CPU version. Hence, shared memory and minimizing global memory accesses significantly improved performance.

2.3 *Cublas* version

cuBLAS, a BLAS function library for CUDA, was used to multiple matrices with *cublasSgemm* function, which performs single-precision general matrix multiplication (GEMM) [2]. We executed the cuBLAS version multiple times and present the results in the Table 3.

Run 1	Run 2	Run 3	Run 4	Run 5	**Avg**
0.93	1.07	1.32	1.52	1.42	**1.25**

Table 3: Execution times for matrix multiplication using cuBLAS over multiple runs

So, the cuBLAS version provides a 13233x speedup over the CPU version, a 140.6x speedup compared to the basic GPU version, and a 12.1x speedup over the tiled GPU. cuBLAS shows essential performance gains over previous versions due to its optimization for GPU hardware.

3 Conclusion

In this TP, we implemented and analyzed three versions of matrix multiplication on a GPU: a basic global memory version, a tiled shared memory version, and a version with cuBLAS.

Our results show that the usage of shared memory in the tiled approach significantly reduces memory latency and leads to a notable speedup over the basic implementation. The cuBLAS implementation outperformed both custom implementations by a large margin. Further experiments could be conducted with mixed-precision arithmetic or tensor cores, to enhance performance even more.

Code could be review following this link.

References

[1] Elisabeth Brunet. Gpu for deep learning - optimized matrix multiplication. Lecture presentation, Telecom SudParis, 2025.

[2] Goran Frehse. Gpu for deep learning - introduction to cublas. Lecture presentation, ENSTA Paris, 2025.